

Domain 3 — Section Review

Claude Code Configuration & Workflows

11/12

92% · Strong · 1 missed

Q1 ✓ **CORRECT**

You want to create a custom `/review` slash command that runs your team's code review checklist. This command should be available to every developer automatically when they clone or pull the repository. Where should you create this command file?

- A** In `~/claude/commands/` in each developer's home directory.
- B** In the `CLAUDE.md` file at the project root as a command definition section.
- C** In `.claude/commands/` in the project repository.
- D** In `.claude/config.json` with a commands array.

Why C is correct:

Path prefix is the whole answer. `~/claude/` is personal, never shared via version control. `.claude/commands/` in the project repository is version-controlled — automatically available to all developers on clone/pull. `CLAUDE.md` is for project instructions, not command definitions. `.claude/config.json` does not exist as a commands mechanism.

Q2 ✓ **CORRECT**

Developer A's Claude Code faithfully follows team conventions. Developer B, on the same branch and repository, gets inconsistent behaviour. Both developers' `/memory` output shows the same configuration files are loaded. Most likely root cause:

- A Developer B hasn't installed the required MCP servers for this project.
- B Developer A wrote the conventions to `~/.claude/CLAUDE.md` (user-level), which git never tracked or committed.**
- C Developer B's `CLAUDE.md` has a YAML syntax error causing silent load failure.
- D A directory-level `CLAUDE.md` in a subdirectory is overriding project-root configuration for Developer B.

Why B is correct:

Divergent teammates on the same repo → suspect user-level vs project-level config first.

`~/.claude/CLAUDE.md` lives in the home directory — git never tracks it. Developer A wrote standards there; git never committed it; Developer B's clone has nothing. `/memory` confirming files are loaded is consistent with this — Developer B's `~/.claude/CLAUDE.md` simply has no relevant content.

Q3 ✓ CORRECT

Your team's `/brainstorm` skill explores design alternatives and produces extensive exploratory output — dozens of options and trade-off analyses. Developers complain this floods their main conversation history, making it hard to focus on implementation afterwards. The fix is:

- A Move the skill to `~/.claude/skills/` so it runs with reduced verbosity.
- B Add `allowed-tools: []` to restrict the skill from making any tool calls.**

C Add context: fork to the skill's SKILL.md frontmatter.



D Split the skill into multiple smaller commands that each produce less output.

Why C is correct:

context: fork = isolated sub-agent context. Verbose output stays in the fork; only the summary returns to the main conversation. Recognition trigger: "produces verbose output" / "clutters main conversation" / "keep main context clean." Moving to ~/.claude/skills/ (A) changes who has access, not output behaviour. allowed-tools (B) restricts tool calls during the skill, not how much it outputs.

Q4 ✓ CORRECT

Your CI pipeline script runs `claude "Analyse this pull request for security issues"` and the job hangs indefinitely waiting for interactive input. The correct fix is:

- A Set the environment variable `CLAUDE_HEADLESS=true` before running.
- B Add the `-p` flag: `claude -p "Analyse this pull request for security issues"`**
- C Redirect stdin from `/dev/null`: `claude "Analyse..." < /dev/null`
- D Add the `--batch` flag: `claude --batch "Analyse this pull request for security issues"`

Why B is correct:

`-p (--print)` is the documented non-interactive flag for CI/CD. It processes the prompt, outputs to stdout, and exits without waiting for user input. `CLAUDE_HEADLESS` (A) doesn't exist. The `/dev/null` redirect (C) doesn't properly address Claude Code's interactive input mechanism. `--batch` (D) is not a valid Claude Code CLI flag.

Q5 ✓ CORRECT

Test files in your codebase are co-located alongside source files (`Button.test.tsx` next to `Button.tsx`) across dozens of directories. You want identical testing conventions applied to all test files regardless of location. Most maintainable approach:

- A A single `CLAUDE.md` at project root under a '## Testing' header, relying on Claude to infer when it applies.
- B Place a `CLAUDE.md` in each directory containing test files.

Create a `.claude/rules/` file with paths: `["**/*.test.*", "**/*.spec.*"]` YAML frontmatter

C containing the testing conventions.



D Create a /generate-tests skill with conventions embedded in SKILL.md.

Why C is correct:

Path-scoped rules for file-type conventions that span many directories. .claude/rules/ with a glob loads automatically when editing matching files — cross-directory, zero manual invocation. Root CLAUDE.md (A) loads for every interaction including non-test work. Per-directory CLAUDE.md (B) requires maintaining N files — unscalable. Skills (D) require explicit invocation and won't fire automatically in CI.

Q6 ✓ **CORRECT**

You are restructuring a monolithic application into microservices — changes across 45+ files, decisions about service boundaries, module dependencies, and infrastructure requirements. Which mode should you use?

A Direct execution with comprehensive upfront instructions detailing how each service should be structured.

B Plan mode — explore the codebase and design an implementation approach before making any changes.

C Direct execution incrementally, letting the implementation reveal natural service boundaries.

D Direct execution, switching to plan mode only if unexpected complexity emerges during implementation.

Why B is correct:

Plan mode for: large-scale changes, multiple valid approaches, architectural decisions, multi-file

modifications. The complexity is already known — it doesn't emerge later. Plan mode enables safe codebase exploration and design before committing to changes, preventing costly rework.   on direct execution (C) risks discovering incompatible decisions late, when rework is most expensive.

Q7 ✓ CORRECT

After a second commit to a PR, the CI review re-flags issues that were already addressed in the first review, generating duplicate comments. How do you fix this?

- A Run the review on only the incremental diff since the last reviewed commit.
- B Include prior review findings in context when re-running, instructing Claude to report only new or still-unaddressed issues.**
- C Use a separate independent Claude instance for each commit review to avoid context contamination.
- D Increase max_tokens so Claude can process the full PR history and de-duplicate internally.

Why B is correct:

Incremental review with prior context injection. Including the previous findings and explicitly instructing Claude to report only new or still-unaddressed issues directly prevents duplicate comments. Running only the diff (A) misses issues that span files not changed in the latest commit. Independent instances (C) won't know what was already reported. max_tokens (D) is irrelevant — the issue is the instruction, not output length.

Q8 ✓ CORRECT

Your CI pipeline runs Claude Code with `--print` mode using CLAUDE.md for project context, and developers find the reviews insightful. However, integrating findings is difficult — Claude produces narrative paragraphs that must be manually copied into PR comments. Your team wants to

automatically post each finding as a separate inline PR comment at the relevant code location requiring structured data with file path, line number, severity, and suggested fix. Which is the most effective approach?



- A** Post-process Claude's narrative output with a regex parser to extract file paths and line numbers.
- B** Add an instruction to CLAUDE.md: 'Always format findings as JSON with file, line, severity, and fix fields.'
- C** Use `--output-format json` with `--json-schema` to enforce structured output containing the required fields for each finding.
- D** Run a second Claude pass that takes the narrative output and converts it to structured JSON.

Why C is correct:

--output-format json with --json-schema enforces machine-parseable structured output from the Claude Code CLI. This is the documented CI/CD approach — structured findings can be directly posted as inline PR comments. Regex parsing (A) is brittle and breaks on format variation. CLAUDE.md instructions (B) are probabilistic and produce inconsistent JSON. A second conversion pass (D) adds cost and latency without reliability guarantees.

Q9 ✓ **CORRECT**

Your pipeline script runs `claude "Analyze this pull request for security issues"` but the job hangs indefinitely. Logs indicate Claude Code is waiting for interactive input. What's the correct approach to run Claude Code in an automated pipeline?

- A** Set the `CLAUDE_NONINTERACTIVE=true` environment variable before running the command.

B Add the `-p` flag: `claude -p "Analyze this pull request for security`  `e: `

C Pipe `/dev/null` to stdin: `claude "Analyze..." < /dev/null`

D Use the `--headless` flag: `claude --headless "Analyze this pull request for security issues"`

Why B is correct:

-p (--print) is the non-interactive flag for CI/CD. It processes the prompt, writes output to stdout, and exits without waiting for user input. `CLAUDE_NONINTERACTIVE` (A) doesn't exist. Piping `/dev/null` (C) doesn't properly address Claude Code's interactive input mechanism. `--headless` (D) is not a valid Claude Code flag.

Q10 ✓ CORRECT

Your CI pipeline runs the Claude Code CLI with `--print` mode using `CLAUDE.md` to provide project context. Developers find the reviews insightful but integrating findings is difficult — Claude produces narrative paragraphs that must be manually copied into PR comments. Your team wants to automatically post each finding as a separate inline PR comment, which requires structured data with file path, line number, severity, and suggested fix. What's the most effective approach?

A Add an instruction to `CLAUDE.md`: 'Always format findings as JSON with file, line, severity, and fix fields.'

B Post-process Claude's narrative output with a regex parser to extract file paths and line numbers.

C Use `--output-format json` with `--json-schema` to enforce structured output containing the required fields for each finding.

Run a second Claude pass that converts the narrative output into structured JSON.

D

**Why C is correct:**

--output-format json with --json-schema is the documented CLI mechanism for enforcing structured output. The schema guarantees machine-parseable findings that map directly to inline PR comment fields. CLAUDE.md instructions (A) are probabilistic — inconsistent JSON results. Regex parsing (B) breaks on any format variation. A second conversion pass (D) adds cost and latency without reliability.

Q11 ✓ **CORRECT**

After an initial automated review generates 12 findings, a developer pushes new commits to address the issues. When the review runs again, it produces 8 findings — but developers report that 5 duplicate earlier comments on code that was already fixed in the new commits. What's the most effective way to eliminate this redundant feedback while maintaining thorough analysis?

A Run the review on only the diff of the new commits, ignoring previously reviewed code.

B Include the prior review findings in context when re-running, and instruct Claude to report only new issues or issues not yet addressed.

C Use a separate independent Claude instance for each review run to avoid anchoring on prior findings.

D Increase `max_tokens` so Claude can process the full PR history and deduplicate findings internally.

Why B is correct:

Inject prior findings as context and instruct Claude to report only what's new or unresolved. This directly prevents duplicate comments. Running only the new diff (A) misses issues that span files unchanged in the latest commits. Independent instances (C) have no knowledge of what was already reported. `max_tokens` (D) is irrelevant — the problem is the missing instruction, not output length.

Q12 X INCORRECT



A developer is implementing a Redis caching layer for the first time. They have a rough idea of what they need but are unfamiliar with cache invalidation strategies, stampede prevention, and failure mode implications. They ask Claude to implement the solution immediately. What iterative technique best ensures these concerns are addressed before any code is written?

- A Direct execution — start implementing and revisit design decisions as complexity emerges during development.
- B Plan mode — explore what a Redis implementation would look like before committing to an approach.
- C The interview pattern — have Claude ask targeted questions to surface cache design considerations before writing code.
- D Test-driven iteration — write cache behaviour tests first, then implement to make them pass.

Why C is correct:

Interview pattern for unfamiliar domains: have Claude ask questions first. The developer doesn't know what they don't know — cache invalidation strategies, stampede prevention (e.g., cache locking), TTL decisions, and failure modes are all non-obvious. The interview pattern surfaces these before a single line is written. Plan mode (B) explores implementation options but doesn't surface the developer's unstated requirements. Test-driven (D) requires the developer to already know the edge cases to test.

Next Section →